



UNIVERSITÀ
DI TRENTO



Freie Universität Bozen
Libera Università di Bolzano
Università Lìdia de Bulsan

Optimization-based Reactive Control

WHERE summer school, Trento

Andrea Del Prete

September 21-27, 2026

University of Trento

Table of contents

1. Introduction
2. QP-based Single-Rigid-Body Control
3. Joint-Space Inverse Dynamics Control
4. From Joint Space to Task Space Control
5. Task Models
6. Extension to legged robots — Under-actuation and contacts
7. Multi-Task Control
8. Computational Aspects

Introduction

A generic constrained optimization problem:

$$\begin{array}{ll} \underset{x}{\text{minimize}} & f(x) \\ \text{subject to} & g(x) = 0 \\ & h(x) \leq 0 \end{array} \quad (1)$$

A generic **constrained optimization problem**:

$$\begin{aligned} & \underset{x}{\text{minimize}} && f(x) \\ & \text{subject to} && g(x) = 0 \\ & && h(x) \leq 0 \end{aligned} \tag{1}$$

Generic constrained optimization problems are **hard** to solve.

A generic **constrained optimization problem**:

$$\begin{aligned} & \underset{x}{\text{minimize}} && f(x) \\ & \text{subject to} && g(x) = 0 \\ & && h(x) \leq 0 \end{aligned} \tag{1}$$

Generic constrained optimization problems are **hard** to solve.

Convex optimization problems are generally easier/faster to solve.

A generic **constrained optimization problem**:

$$\begin{aligned} & \underset{x}{\text{minimize}} && f(x) \\ & \text{subject to} && g(x) = 0 \\ & && h(x) \leq 0 \end{aligned} \tag{1}$$

Generic constrained optimization problems are **hard** to solve.

Convex optimization problems are generally easier/faster to solve.

Some of them can be solved **extremely** fast (e.g., **1 kHz**).

Quadratic and Least-Squares Programs

Least-Squares Programs (LSP) have:

- linear equality/inequality constraints ($Ax \leq b$, or $Ax = b$)
- 2-norm of linear cost function ($\|Ax - b\|^2$)

Quadratic and Least-Squares Programs

Least-Squares Programs (LSP) have:

- linear equality/inequality constraints ($Ax \leq b$, or $Ax = b$)
- 2-norm of linear cost function ($\|Ax - b\|^2$)

LSPs are subclass of convex Quadratic Programs (QPs), which have:

- linear equality/inequality constraints ($Ax \leq b$, or $Ax = b$)
- convex quadratic cost function ($x^\top Hx + h^\top x$, with $H \geq 0$)

Quadratic and Least-Squares Programs

Least-Squares Programs (LSP) have:

- linear equality/inequality constraints ($Ax \leq b$, or $Ax = b$)
- 2-norm of linear cost function ($\|Ax - b\|^2$)

LSPs are subclass of convex Quadratic Programs (QPs), which have:

- linear equality/inequality constraints ($Ax \leq b$, or $Ax = b$)
- convex quadratic cost function ($x^\top Hx + h^\top x$, with $H \geq 0$)

LSPs and convex QPs can be solved extremely fast with off-the-shelf softwares

Quadratic and Least-Squares Programs

Least-Squares Programs (LSP) have:

- linear equality/inequality constraints ($Ax \leq b$, or $Ax = b$)
- 2-norm of linear cost function ($\|Ax - b\|^2$)

LSPs are subclass of convex Quadratic Programs (QPs), which have:

- linear equality/inequality constraints ($Ax \leq b$, or $Ax = b$)
- convex quadratic cost function ($x^T Hx + h^T x$, with $H \geq 0$)

LSPs and convex QPs can be solved extremely fast with off-the-shelf softwares

→ We can solve LSP/QPs inside 1 kHz control loops!

QP-based Single-Rigid-Body Control

Problem Definition

Track a **reference** trajectory for **CoM** $c^{ref}(t)$ and base **orientation** $R^{ref}(t)$ of a legged robot.

Problem Definition

Track a **reference** trajectory for **CoM** $c^{ref}(t)$ and base **orientation** $R^{ref}(t)$ of a legged robot.

Let us work with a **single-rigid-body** model, which captures relationship between **contact forces** and **CoM** motion:

$$\begin{aligned} m\ddot{c} + mg &= \sum_i f_i \\ L_G \dot{\omega} + \omega \times L_G \omega &= \sum_i (p_i - c) \times f_i \end{aligned} \tag{2}$$

Problem Definition

Track a **reference** trajectory for **CoM** $c^{ref}(t)$ and base **orientation** $R^{ref}(t)$ of a legged robot.

Let us work with a **single-rigid-body** model, which captures relationship between **contact forces** and **CoM** motion:

$$\begin{aligned} m\ddot{c} + mg &= \sum_i f_i \\ L_G \dot{\omega} + \omega \times L_G \omega &= \sum_i (p_i - c) \times f_i \end{aligned} \quad (2)$$

Expressed in matrix form:

$$\underbrace{\begin{bmatrix} m\ddot{c} \\ L_G \dot{\omega} \end{bmatrix}}_W + \underbrace{\begin{bmatrix} mg \\ \omega \times L_G \omega \end{bmatrix}}_W = \underbrace{\begin{bmatrix} I & \dots & I \\ (p_0 - c) \times & \dots & (p_{k-1} - c) \times \end{bmatrix}}_{J_{cb}^T} \underbrace{\begin{bmatrix} f_0 \\ \vdots \\ f_{k-1} \end{bmatrix}}_f \quad (3)$$

Pseudo-inverse Approach

Compute a desired value of W with a feedforward + PD feedback term:

$$W^d = \begin{bmatrix} m(\ddot{c}^{ref} + \text{PD}(c^{ref}, c)) + mg \\ L_G(\dot{\omega}^{ref} + \text{PD}(R^{ref}, R)) + \omega \times L_G \omega \end{bmatrix} \quad (4)$$

The term $\omega \times L_G \omega$ is typically neglected.

Pseudo-inverse Approach

Compute a desired value of W with a feedforward + PD feedback term:

$$W^d = \begin{bmatrix} m(\ddot{c}^{ref} + \text{PD}(c^{ref}, c)) + mg \\ L_G(\dot{\omega}^{ref} + \text{PD}(R^{ref}, R)) + \omega \times L_G\omega \end{bmatrix} \quad (4)$$

The term $\omega \times L_G\omega$ is typically neglected.

Then compute the corresponding **contact forces** with a **pseudo-inverse**:

$$f^d = (J_{cb}^\top)^\dagger W^d \quad (5)$$

Pseudo-inverse Approach

Compute a desired value of W with a feedforward + PD feedback term:

$$W^d = \begin{bmatrix} m(\ddot{c}^{ref} + \text{PD}(c^{ref}, c)) + mg \\ L_G(\dot{\omega}^{ref} + \text{PD}(R^{ref}, R)) + \omega \times L_G \omega \end{bmatrix} \quad (4)$$

The term $\omega \times L_G \omega$ is typically neglected.

Then compute the corresponding **contact forces** with a **pseudo-inverse**:

$$f^d = (J_{cb}^\top)^\dagger W^d \quad (5)$$

Finally map the contact forces to **joint torques**:

$$\tau = -J_{cj}^\top f^d \quad (6)$$

Pseudo-inverse Approach

Compute a desired value of W with a feedforward + PD feedback term:

$$W^d = \begin{bmatrix} m(\ddot{c}^{ref} + \text{PD}(c^{ref}, c)) + mg \\ L_G(\dot{\omega}^{ref} + \text{PD}(R^{ref}, R)) + \omega \times L_G\omega \end{bmatrix} \quad (4)$$

The term $\omega \times L_G\omega$ is typically neglected.

Then compute the corresponding **contact forces** with a **pseudo-inverse**:

$$f^d = (J_{cb}^\top)^\dagger W^d \quad (5)$$

Finally map the contact forces to **joint torques**:

$$\tau = -J_{cj}^\top f^d \quad (6)$$

ISSUE: What if the contact forces are outside their **friction cones**?

Quadratic Programming Approach

The **pseudo-inverse** solves the following optimization problem:

$$\underset{f}{\text{minimize}} \|J_{cb}^{\top} f - W^d\|^2 \quad (7)$$

This is an **unconstrained Least-Squares** Program.

Quadratic Programming Approach

The **pseudo-inverse** solves the following optimization problem:

$$\underset{f}{\text{minimize}} \quad \|J_{cb}^{\top} f - W^d\|^2 \quad (7)$$

This is an **unconstrained Least-Squares** Program.

To account for **friction cones** we add **inequality constraints**:

$$\begin{aligned} \underset{f}{\text{minimize}} \quad & \|J_{cb}^{\top} f - W^d\|^2 \\ \text{subject to} \quad & Bf \leq 0 \end{aligned} \quad (8)$$

This is a **constrained Least-Squares** Program.

Quadratic Programming Approach

The **pseudo-inverse** solves the following optimization problem:

$$\underset{f}{\text{minimize}} \quad \|J_{cb}^{\top} f - W^d\|^2 \quad (7)$$

This is an **unconstrained Least-Squares** Program.

To account for **friction cones** we add **inequality constraints**:

$$\begin{aligned} \underset{f}{\text{minimize}} \quad & \|J_{cb}^{\top} f - W^d\|^2 \\ \text{subject to} \quad & Bf \leq 0 \end{aligned} \quad (8)$$

This is a **constrained Least-Squares** Program.

Remark: Friction cones are **quadratic**, but approximate them with **linear** inequalities.

Joint-Space Inverse Dynamics Control

Robot Manipulator

Given (nonlinear) manipulator dynamics:

$$M(q)\dot{v} + h(q, v) = \tau \quad (9)$$

Problem

Find $\tau(t)$ so that $q(t)$ follows reference $q^r(t)$.

Robot Manipulator

Given (nonlinear) manipulator dynamics:

$$M(q)\dot{v} + h(q, v) = \tau \quad (9)$$

Problem

Find $\tau(t)$ so that $q(t)$ follows reference $q^r(t)$.

Assumption

We know dynamics and can measure q and v .

Robot Manipulator

Given (nonlinear) manipulator dynamics:

$$M(q)\dot{v} + h(q, v) = \tau \quad (9)$$

Problem

Find $\tau(t)$ so that $q(t)$ follows reference $q^r(t)$.

Assumption

We know dynamics and can measure q and v .

Solution

Set $\tau = M(q)\dot{v}^d + h(q, v) \rightarrow$ closed-loop dynamics is $\dot{v} = \dot{v}^d$.

Robot Manipulator

Given (nonlinear) manipulator dynamics:

$$M(q)\dot{v} + h(q, v) = \tau \quad (9)$$

Problem

Find $\tau(t)$ so that $q(t)$ follows reference $q^r(t)$.

Assumption

We know dynamics and can measure q and v .

Solution

Set $\tau = M(q)\dot{v}^d + h(q, v) \rightarrow$ closed-loop dynamics is $\dot{v} = \dot{v}^d$.

Select $\dot{v}^d$ so that $q(t)$ follows $q^r(t)$:

Robot Manipulator

Given (nonlinear) manipulator dynamics:

$$M(q)\dot{v} + h(q, v) = \tau \quad (9)$$

Problem

Find $\tau(t)$ so that $q(t)$ follows reference $q^r(t)$.

Assumption

We know dynamics and can measure q and v .

Solution

Set $\tau = M(q)\dot{v}^d + h(q, v) \rightarrow$ closed-loop dynamics is $\dot{v} = \dot{v}^d$.

Select $\dot{v}^d$ so that $q(t)$ follows $q^r(t)$:

$$\dot{v}^d = \dot{v}^r$$

Robot Manipulator

Given (nonlinear) manipulator dynamics:

$$M(q)\dot{v} + h(q, v) = \tau \quad (9)$$

Problem

Find $\tau(t)$ so that $q(t)$ follows reference $q^r(t)$.

Assumption

We know dynamics and can measure q and v .

Solution

Set $\tau = M(q)\dot{v}^d + h(q, v) \rightarrow$ closed-loop dynamics is $\dot{v} = \dot{v}^d$.

Select $\dot{v}^d$ so that $q(t)$ follows $q^r(t)$:

$$\dot{v}^d = \dot{v}^r - K_d(v - v^r) - K_p(q - q^r) \quad (10)$$

where K_p, K_d are diagonal positive-definite gain matrices.

Convergence

Show that $q(t)$ converges to $q^r(t)$.

Convergence

Show that $q(t)$ converges to $q^r(t)$.

Closed-loop dynamics is

$$\dot{v} = \dot{v}^r - K_d \underbrace{(v - v^r)}_{\dot{e}} - K_p \underbrace{(q - q^r)}_e$$

Convergence

Show that $q(t)$ converges to $q^r(t)$.

Closed-loop dynamics is

$$\dot{v} = \dot{v}^r - K_d \underbrace{(v - v^r)}_{\dot{e}} - K_p \underbrace{(q - q^r)}_e$$

$$\ddot{e} = -K_d \dot{e} - K_p e$$

Convergence

Show that $q(t)$ converges to $q^r(t)$.

Closed-loop dynamics is

$$\dot{v} = \dot{v}^r - K_d \underbrace{(v - v^r)}_{\dot{e}} - K_p \underbrace{(q - q^r)}_e$$

$$\ddot{e} = -K_d \dot{e} - K_p e$$

$$\underbrace{\begin{bmatrix} \dot{e} \\ \ddot{e} \end{bmatrix}}_{\dot{x}} = \underbrace{\begin{bmatrix} 0 & I \\ -K_p & -K_d \end{bmatrix}}_A \underbrace{\begin{bmatrix} e \\ \dot{e} \end{bmatrix}}_x$$

Convergence

Show that $q(t)$ converges to $q^r(t)$.

Closed-loop dynamics is

$$\begin{aligned}\dot{v} &= \dot{v}^r - K_d \underbrace{(v - v^r)}_{\dot{e}} - K_p \underbrace{(q - q^r)}_e \\ \ddot{e} &= -K_d \dot{e} - K_p e \\ \underbrace{\begin{bmatrix} \dot{e} \\ \ddot{e} \end{bmatrix}}_{\dot{x}} &= \underbrace{\begin{bmatrix} 0 & I \\ -K_p & -K_d \end{bmatrix}}_A \underbrace{\begin{bmatrix} e \\ \dot{e} \end{bmatrix}}_x\end{aligned}$$

A is Hurwitz if K_p and K_d are diagonal and positive-definite $\rightarrow$
 $\lim_{t \rightarrow \infty} x(t) = 0 \rightarrow \lim_{t \rightarrow \infty} q(t) = q^r(t)$

Many names for the same approach

This control law:

$$\tau = M(\ddot{v}^r - K_d \dot{e} - K_p e) + h \quad (11)$$

is known as:

- **Inverse-Dynamics (ID) Control**: because based on inverse dynamics computation.
- **Computed Torque**: because it computes torques needed to get desired accelerations.
- **Feedback Linearization** (from control theory): because it uses state feedback to linearize closed-loop dynamics.

Inverse Dynamics (ID) Control as Least-Squares Problem

Solution of optimization problem:

$$\begin{aligned} (\tau^*, \dot{v}^*) = & \underset{\tau, \dot{v}}{\operatorname{argmin}} \quad ||\dot{v} - \dot{v}^d||^2 \\ & \text{subject to} \quad M\dot{v} + h = \tau \end{aligned} \tag{12}$$

with $\dot{v}^d = \dot{v}^r - K_d \dot{e} - K_p e$

Inverse Dynamics (ID) Control as Least-Squares Problem

Solution of optimization problem:

$$\begin{aligned} (\tau^*, \dot{v}^*) = \underset{\tau, \dot{v}}{\operatorname{argmin}} \quad & ||\dot{v} - \dot{v}^d||^2 \\ \text{subject to} \quad & M\dot{v} + h = \tau \end{aligned} \tag{12}$$

with $\dot{v}^d = \dot{v}^r - K_d \dot{e} - K_p e$, is exactly the ID control law:

$$\tau^* = M\dot{v}^d + h, \tag{13}$$

Inverse Dynamics (ID) Control as Least-Squares Problem

Solution of optimization problem:

$$\begin{aligned} (\tau^*, \dot{v}^*) = & \underset{\tau, \dot{v}}{\operatorname{argmin}} \quad ||\dot{v} - \dot{v}^d||^2 \\ & \text{subject to} \quad M\dot{v} + h = \tau \end{aligned} \quad (12)$$

with $\dot{v}^d = \dot{v}^r - K_d \dot{e} - K_p e$, is exactly the ID control law:

$$\tau^* = M\dot{v}^d + h, \quad (13)$$

No advantage in solving (12) to compute (13), **but** (12) is starting point to solve more complex problems.

Inverse Dynamics (ID) Control as Least-Squares Problem

Solution of optimization problem:

$$\begin{aligned} (\tau^*, \dot{v}^*) = \underset{\tau, \dot{v}}{\operatorname{argmin}} \quad & \|\dot{v} - \dot{v}^d\|^2 \\ \text{subject to} \quad & M\dot{v} + h = \tau \end{aligned} \quad (12)$$

with $\dot{v}^d = \dot{v}^r - K_d \dot{e} - K_p e$, is exactly the ID control law:

$$\tau^* = M\dot{v}^d + h, \quad (13)$$

No advantage in solving (12) to compute (13), **but** (12) is starting point to solve more complex problems.

Problem (12) is a Least-Squares Program.

Adding Torque Limits to ID Control

Take the ID control LSP:

$$\begin{aligned} & \underset{\tau, \dot{v}}{\text{minimize}} && ||\dot{v} - \dot{v}^d||^2 \\ & \text{subject to} && M\dot{v} + h = \tau \end{aligned} \tag{14}$$

Adding Torque Limits to ID Control

Take the ID control LSP:

$$\begin{aligned} & \underset{\tau, \dot{v}}{\text{minimize}} && ||\dot{v} - \dot{v}^d||^2 \\ & \text{subject to} && M\dot{v} + h = \tau \end{aligned} \tag{14}$$

LSPs allow for **linear inequality constraints** $\rightarrow$ we can add **torque limits**:

$$\begin{aligned} & \underset{\tau, \dot{v}}{\text{minimize}} && ||\dot{v} - \dot{v}^d||^2 \\ & \text{subject to} && M\dot{v} + h = \tau \\ & && \tau^{\min} \leq \tau \leq \tau^{\max} \end{aligned} \tag{15}$$

Adding Torque Limits to ID Control

Take the ID control LSP:

$$\begin{aligned} & \underset{\tau, \dot{v}}{\text{minimize}} && ||\dot{v} - \dot{v}^d||^2 \\ & \text{subject to} && M\dot{v} + h = \tau \end{aligned} \tag{14}$$

LSPs allow for **linear inequality constraints** → we can add **torque limits**:

$$\begin{aligned} & \underset{\tau, \dot{v}}{\text{minimize}} && ||\dot{v} - \dot{v}^d||^2 \\ & \text{subject to} && M\dot{v} + h = \tau \\ & && \tau^{\min} \leq \tau \leq \tau^{\max} \end{aligned} \tag{15}$$

Main advantage of optimization: inequality constraints.

Adding Current Limits for Electric Motors

In electric motors current i is proportional to torque τ :

$$i = k_{\tau}\tau \tag{16}$$

Adding Current Limits for Electric Motors

In electric motors current i is proportional to torque τ :

$$i = k_{\tau}\tau \quad (16)$$

Add **current limits**:

$$\begin{aligned} & \underset{\tau, \dot{v}}{\text{minimize}} && ||\dot{v} - \dot{v}^d||^2 \\ & \text{subject to} && M\dot{v} + h = \tau \\ & && \tau^{min} \leq \tau \leq \tau^{max} \\ & && i^{min} \leq k_{\tau}\tau \leq i^{max} \end{aligned} \quad (17)$$

Adding Joint Velocity Limits

Assuming **constant accelerations** $\dot{v}$ during time step Δt :

$$v(t + \Delta t) = v(t) + \Delta t \dot{v} \quad (18)$$

Adding Joint Velocity Limits

Assuming **constant accelerations** $\dot{v}$ during time step Δt :

$$v(t + \Delta t) = v(t) + \Delta t \dot{v} \quad (18)$$

Add joint **velocity limits**:

$$\begin{aligned} & \underset{\tau, \dot{v}}{\text{minimize}} \quad \|\dot{v} - \dot{v}^d\|^2 \\ & \text{subject to} \quad M\dot{v} + h = \tau \\ & \quad \tau^{\min} \leq \tau \leq \tau^{\max} \\ & \quad \dot{v}^{\min} \leq k_{\tau}\tau \leq \dot{v}^{\max} \\ & \quad v^{\min} \leq v + \Delta t\dot{v} \leq v^{\max} \end{aligned} \quad (19)$$

Adding Joint Position Limits

Could use same trick for position limits:

$$q(t + \Delta t) = q(t) + \Delta t \, v(t) + \frac{1}{2} \Delta t^2 \dot{v} \quad (20)$$

Adding Joint Position Limits

Could use same trick for position limits:

$$q(t + \Delta t) = q(t) + \Delta t v(t) + \frac{1}{2} \Delta t^2 \dot{v} \quad (20)$$

However, this can result in **high accelerations**, typically **incompatible** with torque/current limits $\rightarrow$ unfeasible LSP.

Adding Joint Position Limits

Could use same trick for position limits:

$$q(t + \Delta t) = q(t) + \Delta t v(t) + \frac{1}{2} \Delta t^2 \dot{v} \quad (20)$$

However, this can result in **high accelerations**, typically **incompatible** with torque/current limits $\rightarrow$ unfeasible LSP.

Better approaches exist [1, 7, 2], but we don't discuss them here.

From Joint Space to Task Space Control

Limits of Joint-Space Control

Joint-space control needs reference **joint** trajectory $q^r(t)$.

Limits of Joint-Space Control

Joint-space control needs reference **joint** trajectory $q^r(t)$.

What if we have reference trajectory $x^r(t)$ for **end-effector**?

Option 1: Mapping End-Effector Space to Joint Space

Compute joint trajectory $q^r(t)$ corresponding to $x^r(t)$, then apply joint-space control:

Option 1: Mapping End-Effector Space to Joint Space

Compute joint trajectory $q^r(t)$ corresponding to $x^r(t)$, then apply joint-space control:

$$\text{Find } q^r(t) \text{ such that } FG(q^r(t)) = x^r(t) \quad \forall t \in [0, T],$$

Option 1: Mapping End-Effector Space to Joint Space

Compute joint trajectory $q^r(t)$ corresponding to $x^r(t)$, then apply joint-space control:

$$\begin{aligned} \text{Find } q^r(t) \text{ such that } FG(q^r(t)) &= x^r(t) & \forall t \in [0, T], \\ \rightarrow q^r(t) &= FG^\dagger(x^r(t)) & \forall t \in [0, T], \end{aligned} \quad (21)$$

where:

- $FG(.) \triangleq$ forward geometry function of end-effector
- $FG^\dagger(.)$ is such that $FG(FG^\dagger(x)) = x, \forall x$

Option 1: Mapping End-Effector Space to Joint Space

Compute joint trajectory $q^r(t)$ corresponding to $x^r(t)$, then apply joint-space control:

$$\begin{aligned} \text{Find } q^r(t) \quad \text{such that} \quad FG(q^r(t)) &= x^r(t) & \forall t \in [0, T], \\ \rightarrow q^r(t) &= FG^\dagger(x^r(t)) & \forall t \in [0, T], \end{aligned} \quad (21)$$

where:

- $FG(.) \triangleq$ forward geometry function of end-effector
- $FG^\dagger(.)$ is such that $FG(FG^\dagger(x)) = x, \forall x$

ISSUES

Problem (21) is challenging (**Inverse Geometry**, nonconvex problem with infinitely many solutions).

Option 1: Mapping End-Effector Space to Joint Space

Compute joint trajectory $q^r(t)$ corresponding to $x^r(t)$, then apply joint-space control:

$$\begin{aligned} \text{Find } q^r(t) \quad \text{such that} \quad FG(q^r(t)) &= x^r(t) & \forall t \in [0, T], \\ \rightarrow \quad q^r(t) &= FG^\dagger(x^r(t)) & \forall t \in [0, T], \end{aligned} \quad (21)$$

where:

- $FG(.) \triangleq$ forward geometry function of end-effector
- $FG^\dagger(.)$ is such that $FG(FG^\dagger(x)) = x, \forall x$

ISSUES

Problem (21) is challenging (**Inverse Geometry**, nonconvex problem with infinitely many solutions).

Tracking $q^r(t)$ is **sufficient but not necessary** to track $x^r(t)$: controller rejects also perturbations affecting q without affecting $FG(q)$.

Option 2: End-Effector Control

Feedback directly end-effector configuration:

$$\ddot{x}^d = \ddot{x}^r - K_d(\dot{x} - \dot{x}^r) - K_p(x - x^r) \quad (22)$$

Option 2: End-Effector Control

Feedback directly end-effector configuration:

$$\ddot{x}^d = \ddot{x}^r - K_d(\dot{x} - \dot{x}^r) - K_p(x - x^r) \quad (22)$$

Differentiate relationship between q and x :

$$\dot{x} = FG(q)\dot{q}$$

Option 2: End-Effector Control

Feedback directly end-effector configuration:

$$\ddot{x}^d = \ddot{x}^r - K_d(\dot{x} - \dot{x}^r) - K_p(x - x^r) \quad (22)$$

Differentiate relationship between q and x :

$$\begin{aligned} x &= FG(q) \\ \dot{x} &= \frac{d}{dt} FG(q) = \underbrace{\frac{\partial FG}{\partial q}}_J \frac{dq}{dt} = Jv \end{aligned}$$

Option 2: End-Effector Control

Feedback directly end-effector configuration:

$$\ddot{x}^d = \ddot{x}^r - K_d(\dot{x} - \dot{x}^r) - K_p(x - x^r) \quad (22)$$

Differentiate relationship between q and x :

$$\begin{aligned} x &= FG(q) \\ \dot{x} &= \frac{d}{dt} FG(q) = \underbrace{\frac{\partial FG}{\partial q}}_J \frac{dq}{dt} = Jv \end{aligned} \quad (23)$$

$$\ddot{x} = J\dot{v} + \dot{J}v$$

Option 2: End-Effector Control

Feedback directly end-effector configuration:

$$\ddot{x}^d = \ddot{x}^r - K_d(\dot{x} - \dot{x}^r) - K_p(x - x^r) \quad (22)$$

Differentiate relationship between q and x :

$$\begin{aligned} x &= FG(q) \\ \dot{x} &= \frac{d}{dt} FG(q) = \underbrace{\frac{\partial FG}{\partial q}}_J \frac{dq}{dt} = Jv \end{aligned} \quad (23)$$

$$\ddot{x} = J\dot{v} + \dot{J}v$$

Desired accelerations should be:

$$\dot{v}^d = J^\dagger(\ddot{x}^d - \dot{J}v) \quad (24)$$

Option 2: End-Effector Control

Feedback directly end-effector configuration:

$$\ddot{x}^d = \ddot{x}^r - K_d(\dot{x} - \dot{x}^r) - K_p(x - x^r) \quad (22)$$

Differentiate relationship between q and x :

$$\begin{aligned} x &= FG(q) \\ \dot{x} &= \frac{d}{dt} FG(q) = \underbrace{\frac{\partial FG}{\partial q}}_J \frac{dq}{dt} = J\dot{v} \end{aligned} \quad (23)$$

$$\ddot{x} = J\ddot{v} + \dot{J}\dot{v}$$

Desired accelerations should be:

$$\dot{v}^d = J^\dagger(\ddot{x}^d - \dot{J}\dot{v}) \quad (24)$$

Finally compute joint torques as:

$$\tau = M\dot{v}^d + h \quad (25)$$

Option 1 VS Option 2

To summarize, both options compute joint torques as:

$$\tau = M\dot{v}^d + h \quad (26)$$

To summarize, both options compute joint torques as:

$$\tau = M\dot{v}^d + h \quad (26)$$

Option 1 computes $\dot{v}^d$ as:

$$\dot{v}^d = \dot{v}^r - PD(q - FG^\dagger(x^r))$$

FG is “inverted” at configuration level.

Option 1 VS Option 2

To summarize, both options compute joint torques as:

$$\tau = M\dot{v}^d + h \quad (26)$$

Option 1 computes $\dot{v}^d$ as:

$$\dot{v}^d = \dot{v}^r - PD(q - FG^\dagger(x^r))$$

FG is “inverted” at configuration level.

Option 2 computes $\dot{v}^d$ as:

$$\dot{v}^d = J^\dagger(\ddot{x}^r - PD(x - x^r) - \dot{J}v)$$

FG is “inverted” at acceleration level.

Option 1 VS Option 2

Option 2 typically preferred:

Option 1 VS Option 2

Option 2 typically preferred:

- + Gains defined in Cartesian space

Option 1 VS Option 2

Option 2 typically preferred:

- + Gains defined in Cartesian space
- + No pre-computations

Option 1 VS Option 2

Option 2 typically preferred:

- + Gains defined in Cartesian space
- + No pre-computations
- + Online specification of reference trajectory

Option 1 VS Option 2

Option 2 typically preferred:

- + Gains defined in Cartesian space
- + No pre-computations
- + Online specification of reference trajectory
- More complex controller

End-effector control law (Option 2):

$$\begin{aligned}\tau &= M\dot{v}^d + h \\ \dot{v}^d &= J^\dagger(\ddot{x}^d - \dot{J}v) \\ \ddot{x}^d &= \ddot{x}^r - PD(x - x^r)\end{aligned}\tag{27}$$

End-effector control law (Option 2):

$$\begin{aligned}\tau &= M\dot{v}^d + h \\ \dot{v}^d &= J^\dagger(\ddot{x}^d - \dot{J}v) \\ \ddot{x}^d &= \ddot{x}^r - PD(x - x^r)\end{aligned}\tag{27}$$

can be computed as:

$$\begin{aligned}\underset{\tau, \dot{v}}{\text{minimize}} \quad & \|J\dot{v} + \dot{J}v - \ddot{x}^d\|^2 \\ \text{subject to} \quad & M\dot{v} + h = \tau\end{aligned}\tag{28}$$

Task Models

Task-Function Approach

Generalize concept of end-effector with **Task**.

Task-Function Approach

Generalize concept of end-effector with **Task**.

Task = control objective.

Task-Function Approach

Generalize concept of end-effector with **Task**.

Task = control objective.

Describe tasks as functions e to minimize (as in optimal control).

Task-Function Approach

Generalize concept of end-effector with **Task**.

Task = control objective.

Describe tasks as functions e to minimize (as in optimal control).

Assume e measures **error** between **real** and **reference** output $y \in \mathbb{R}^m$:

$$\underbrace{e(x, u, t)}_{\text{error}} = \underbrace{y(x, u)}_{\text{real}} - \underbrace{y^*(t)}_{\text{reference}}$$

Task-Function Approach

Generalize concept of end-effector with **Task**.

Task = control objective.

Describe tasks as functions e to minimize (as in optimal control).

Assume e measures **error** between **real** and **reference** output $y \in \mathbb{R}^m$:

$$\underbrace{e(x, u, t)}_{\text{error}} = \underbrace{y(x, u)}_{\text{real}} - \underbrace{y^*(t)}_{\text{reference}}$$

N.B.

Here: e depends on instantaneous state-control value.

In **optimal control**: e depends on state-control trajectory.

IDEA

Given $e(x, u, t)$, find **affine** function of $\dot{v}$ and u to minimize.

Task-Function Types

IDEA

Given $e(x, u, t)$, find **affine** function of $\dot{v}$ and u to minimize.

Three kinds of task functions:

- Affine functions of u : $e(u, t) = A_u u - a(t)$
- Nonlinear functions of v : $e(v, t) = y(v) - y^*(t)$
- Nonlinear functions of q : $e(q, t) = y(q) - y^*(t)$

Task-Function Types

IDEA

Given $e(x, u, t)$, find **affine** function of $\dot{v}$ and u to minimize.

Three kinds of task functions:

- Affine functions of u : $e(u, t) = A_u u - a(t)$
- Nonlinear functions of v : $e(v, t) = y(v) - y^*(t)$
- Nonlinear functions of q : $e(q, t) = y(q) - y^*(t)$

Issue

q and v are not variables in Inverse Dynamics LSP.

Task-Function Types

IDEA

Given $e(x, u, t)$, find **affine** function of $\dot{v}$ and u to minimize.

Three kinds of task functions:

- Affine functions of u : $e(u, t) = A_u u - a(t)$
- Nonlinear functions of v : $e(v, t) = y(v) - y^*(t)$
- Nonlinear functions of q : $e(q, t) = y(q) - y^*(t)$

Issue

q and v are not variables in Inverse Dynamics LSP.

Solution

Impose dynamics of $e(x, t)$ (e.g., $\dot{e} = \dots$)

which should be affine function of $\dot{v}$

such that $\lim_{t \rightarrow \infty} e(x, t) = 0$

Velocity Task-Function

Consider task function: $e(v, t) = y(v) - y^*(t)$.

Velocity Task-Function

Consider task function: $e(v, t) = y(v) - y^*(t)$.

Impose **first-order** linear dynamic:

$$\dot{e} = -Ke$$

Velocity Task-Function

Consider task function: $e(v, t) = y(v) - y^*(t)$.

Impose **first-order** linear dynamic:

$$\begin{aligned} \dot{e} &= -Ke \\ \underbrace{\frac{\partial y}{\partial v}}_{\text{Jacobian}} \dot{v} - \dot{y}^* &= -Ke \end{aligned}$$

Velocity Task-Function

Consider task function: $e(v, t) = y(v) - y^*(t)$.

Impose **first-order** linear dynamic:

$$\begin{aligned}\dot{e} &= -Ke \\ \underbrace{\frac{\partial y}{\partial v}}_{\text{Jacobian}} \dot{v} - \dot{y}^* &= -Ke \\ \underbrace{J}_{A_v} \dot{v} &= \underbrace{\dot{y}^* - Ke}_a\end{aligned}\tag{29}$$

Velocity Task-Function

Consider task function: $e(v, t) = y(v) - y^*(t)$.

Impose **first-order** linear dynamic:

$$\begin{aligned} \dot{e} &= -Ke \\ \underbrace{\frac{\partial y}{\partial v}}_{\text{Jacobian}} \dot{v} - \dot{y}^* &= -Ke \\ \underbrace{J}_{A_v} \dot{v} &= \underbrace{\dot{y}^* - Ke}_a \end{aligned} \tag{29}$$

We got **affine** function of $\dot{v}$.

Velocity Task-Function

Consider task function: $e(v, t) = y(v) - y^*(t)$.

Impose **first-order** linear dynamic:

$$\begin{aligned}\dot{e} &= -Ke \\ \underbrace{\frac{\partial y}{\partial v}}_{\text{Jacobian}} \dot{v} - \dot{y}^* &= -Ke \\ \underbrace{J}_{A_v} \dot{v} &= \underbrace{\dot{y}^* - Ke}_a\end{aligned}\tag{29}$$

We got **affine** function of $\dot{v}$.

N.B.

Could also impose nonlinear dynamics, but linear is ok for most cases.

Consider task function: $e(q, t) = y(q) - y^*(t)$.

Configuration Task-Function

Consider task function: $e(q, t) = y(q) - y^*(t)$.

Impose **second-order** linear dynamics:

$$\ddot{e} = -Ke - D\dot{e}$$

Configuration Task-Function

Consider task function: $e(q, t) = y(q) - y^*(t)$.

Impose **second-order** linear dynamics:

$$\ddot{e} = -Ke - D\dot{e}$$

$$J\dot{v} + \dot{J}v - \ddot{y}^* = -Ke - D\dot{e}$$

Configuration Task-Function

Consider task function: $e(q, t) = y(q) - y^*(t)$.

Impose **second-order** linear dynamics:

$$\begin{aligned}\ddot{e} &= -Ke - D\dot{e} \\ J\dot{v} + \dot{J}v - \ddot{y}^* &= -Ke - D\dot{e} \\ \underbrace{J}_{A_v} \dot{v} &= \underbrace{\ddot{y}^* - \dot{J}v - Ke - D\dot{e}}_a\end{aligned}\tag{30}$$

Configuration Task-Function

Consider task function: $e(q, t) = y(q) - y^*(t)$.

Impose **second-order** linear dynamics:

$$\begin{aligned}\ddot{e} &= -Ke - D\dot{e} \\ J\dot{v} + \dot{J}v - \ddot{y}^* &= -Ke - D\dot{e} \\ \underbrace{J}_{A_v} \dot{v} &= \underbrace{\ddot{y}^* - \dot{J}v - Ke - D\dot{e}}_a\end{aligned}\tag{30}$$

We got **affine** function of $\dot{v}$.

Configuration Task-Function

Consider task function: $e(q, t) = y(q) - y^*(t)$.

Impose **second-order** linear dynamics:

$$\begin{aligned}\ddot{e} &= -Ke - D\dot{e} \\ J\dot{v} + \dot{J}v - \ddot{y}^* &= -Ke - D\dot{e} \\ \underbrace{J}_{A_v} \dot{v} &= \underbrace{\ddot{y}^* - \dot{J}v - Ke - D\dot{e}}_a\end{aligned}\tag{30}$$

We got **affine** function of $\dot{v}$.

N.B.

Could also impose nonlinear dynamics, but linear is ok for most cases.

Task-Function Types: Summary

Functions of $u \rightarrow$ affine.

Task-Function Types: Summary

Functions of $u \rightarrow$ **affine**.

Functions of $x \rightarrow$ **nonlinear**, but cannot be directly imposed.

Task-Function Types: Summary

Functions of $u \rightarrow$ **affine**.

Functions of $x \rightarrow$ **nonlinear**, but cannot be directly imposed.

- For functions of v impose first derivative.

Task-Function Types: Summary

Functions of $u \rightarrow$ **affine**.

Functions of $x \rightarrow$ **nonlinear**, but cannot be directly imposed.

- For functions of v impose first derivative.
- For functions of q impose second derivative.

Task-Function Types: Summary

Functions of $u \rightarrow$ **affine**.

Functions of $x \rightarrow$ **nonlinear**, but cannot be directly imposed.

- For functions of v impose first derivative.
- For functions of q impose second derivative.

End up with **affine** function of $\dot{v}$ and u :

$$g(z) \triangleq \underbrace{\begin{bmatrix} A_v & A_u \end{bmatrix}}_A \underbrace{\begin{bmatrix} \dot{v} \\ u \end{bmatrix}}_z - a$$

Extension to legged robots — Under-actuation and contacts

Find τ that minimizes task function:

$$\begin{aligned} & \underset{z=(\dot{v}, \tau)}{\text{minimize}} && ||Az - a||^2 \\ & \text{subject to} && \begin{bmatrix} M & -I \end{bmatrix} z = -h \end{aligned} \tag{31}$$

Under-actuated systems

Examples:

- legged robots
- wheeled robots
- flying robots
- under-water robots

Under-actuated systems

Examples:

- legged robots
- wheeled robots
- flying robots
- under-water robots

$$\begin{aligned} & \underset{z=(\dot{v}, \tau)}{\text{minimize}} && \|Az - a\|^2 \\ & \text{subject to} && \begin{bmatrix} M & -S^\top \end{bmatrix} z = -h \end{aligned} \tag{32}$$

If system in **contact** $\rightarrow$ account for contact forces f .

If system in **contact** $\rightarrow$ account for contact forces f .

If contacts are **soft**, use estimated forces $\hat{f}$:

$$\begin{aligned} & \underset{z=(\dot{v}, \tau)}{\text{minimize}} && \|Az - a\|^2 \\ & \text{subject to} && \begin{bmatrix} M & -S^\top \end{bmatrix} z = -h + J^\top \hat{f} \end{aligned} \tag{33}$$

TSID for Robots in Rigid Contact

Rigid contacts constrain motion.

$$c(q) = \text{const} \quad \Longleftrightarrow \quad \text{Contact points do not move}$$

TSID for Robots in Rigid Contact

Rigid contacts constrain motion.

$c(q) = \text{const}$ $\iff$ Contact points do not move

$J_c v = 0$ $\iff$ Contact point velocities are null

$J_c \dot{v} + \dot{J}_c v = 0$ $\iff$ Contact point accelerations are null

TSID for Robots in Rigid Contact

Rigid contacts **constrain** motion.

$c(q) = \text{const} \quad \Longleftrightarrow \quad \text{Contact points do not move}$

$J_c v = 0 \quad \Longleftrightarrow \quad \text{Contact point velocities are null}$

$J_c \dot{v} + \dot{J}_c v = 0 \quad \Longleftrightarrow \quad \text{Contact point accelerations are null}$

Introduce forces and constraints:

$$\begin{aligned} & \underset{z=(\dot{v}, f, \tau)}{\text{minimize}} \quad \|Az - a\|^2 \\ & \text{subject to} \quad \begin{bmatrix} J_c & 0 & 0 \\ M & -J_c^\top & -S^\top \end{bmatrix} z = \begin{bmatrix} -j_v \\ -h \end{bmatrix} \end{aligned} \quad (34)$$

Inequality Constraints

Benefit of optimization: **inequality constraints**.

Benefit of optimization: **inequality constraints**.

Any inequality that's **affine** in $z = (\tau, f, \dot{v})$:

- joint torque bounds: $\tau^{min} \leq \tau \leq \tau^{max}$
- (linearized) force friction cones: $Bf \leq 0$
- joint bounds: $\dot{v}^{min} \leq \dot{v} \leq \dot{v}^{max}$
- collision avoidance (more complicated)

Multi-Task Control

Complex robots are **redundant** w.r.t. task they perform

Complex robots are **redundant** w.r.t. task they perform:

- 7-DoF manipulator that controls end-effector placement (6 DoFs)
has 1 DoF of redundancy

Complex robots are **redundant** w.r.t. task they perform:

- 7-DoF manipulator that controls end-effector placement (6 DoFs) has 1 DoF of redundancy
- 18-DoF biped that controls placement of two feet (12 DoFs) has 6 DoFs of redundancy

Complex robots are **redundant** w.r.t. task they perform:

- 7-DoF manipulator that controls end-effector placement (6 DoFs) has 1 DoF of redundancy
- 18-DoF biped that controls placement of two feet (12 DoFs) has 6 DoFs of redundancy

Can use redundancy to execute **secondary** tasks, but how?

Weighted Multi-Objective Optimization

N tasks, each defined by task function

$$g_i(z) = \|A_i z - a_i\|^2 \quad i = 1 \dots N$$

Weighted Multi-Objective Optimization

N tasks, each defined by task function

$$g_i(z) = \|A_i z - a_i\|^2 \quad i = 1 \dots N$$

Simplest strategy: sum functions using **user-defined weights** w_i :

$$\begin{array}{ll} \underset{z=(\dot{v}, f, \tau)}{\text{minimize}} & \sum_{i=1}^N w_i g_i(z) \\ \text{subject to} & \begin{bmatrix} J & 0 & 0 \\ M & -J^\top & -S^\top \end{bmatrix} z = \begin{bmatrix} -j_v \\ -h \end{bmatrix} \end{array}$$

Weighted Multi-Objective Optimization

N tasks, each defined by task function

$$g_i(z) = \|A_i z - a_i\|^2 \quad i = 1 \dots N$$

Simplest strategy: sum functions using **user-defined weights** w_i :

$$\begin{aligned} & \underset{z=(\dot{v}, f, \tau)}{\text{minimize}} && \sum_{i=1}^N w_i g_i(z) \\ & \text{subject to} && \begin{bmatrix} J & 0 & 0 \\ M & -J^\top & -S^\top \end{bmatrix} z = \begin{bmatrix} -j_v \\ -h \end{bmatrix} \end{aligned}$$

PROS Problem remains **computationally-efficient** LSP.

Weighted Multi-Objective Optimization

N tasks, each defined by task function

$$g_i(z) = \|A_i z - a_i\|^2 \quad i = 1 \dots N$$

Simplest strategy: sum functions using **user-defined weights** w_i :

$$\begin{aligned} & \underset{z=(\dot{v}, f, \tau)}{\text{minimize}} && \sum_{i=1}^N w_i g_i(z) \\ & \text{subject to} && \begin{bmatrix} J & 0 & 0 \\ M & -J^\top & -S^\top \end{bmatrix} z = \begin{bmatrix} -j_v \\ -h \end{bmatrix} \end{aligned}$$

PROS Problem remains **computationally-efficient** LSP.

CONS Hard to find weights $\rightarrow$ too large/small weights lead to **numerical issues**.

Hierarchical Multi-Objective Optimization

Alternative: order tasks according to **priority**

Hierarchical Multi-Objective Optimization

Alternative: order tasks according to **priority**

- task 1 more important than task 2
- ...
- task N-1 more important than task N

Hierarchical Multi-Objective Optimization

Alternative: order tasks according to **priority**

- task 1 more important than task 2
- ...
- task N-1 more important than task N

Solve sequence (**cascade**) of N problems, from $i = 1$:

$$\begin{aligned} g_i^* &= \underset{z=(\dot{v}, f, \tau)}{\text{minimize}} && g_i(z) \\ \text{subject to} &&& \begin{bmatrix} J & 0 & 0 \\ M & -J^\top & -S^\top \end{bmatrix} z = \begin{bmatrix} -j_v \\ -h \end{bmatrix} \\ &&& g_j(z) = g_j^* \quad \forall j < i \end{aligned}$$

Hierarchical Multi-Objective Optimization

Alternative: order tasks according to **priority**

- task 1 more important than task 2
- ...
- task N-1 more important than task N

Solve sequence (**cascade**) of N problems, from $i = 1$:

$$\begin{aligned} g_i^* &= \underset{z=(\dot{v}, f, \tau)}{\text{minimize}} && g_i(z) \\ \text{subject to} &&& \begin{bmatrix} J & 0 & 0 \\ M & -J^\top & -S^\top \end{bmatrix} z = \begin{bmatrix} -j_v \\ -h \end{bmatrix} \\ &&& g_j(z) = g_j^* \quad \forall j < i \end{aligned}$$

PROS **Easier** to find priorities than weights.

Hierarchical Multi-Objective Optimization

Alternative: order tasks according to **priority**

- task 1 more important than task 2
- ...
- task N-1 more important than task N

Solve sequence (**cascade**) of N problems, from $i = 1$:

$$\begin{aligned} g_i^* &= \underset{z=(\dot{v}, f, \tau)}{\text{minimize}} && g_i(z) \\ \text{subject to} &&& \begin{bmatrix} J & 0 & 0 \\ M & -J^\top & -S^\top \end{bmatrix} z = \begin{bmatrix} -j_v \\ -h \end{bmatrix} \\ &&& g_j(z) = g_j^* \quad \forall j < i \end{aligned}$$

PROS Easier to find priorities than weights.

CONS More **computationally expensive** to solve several LSPs.

Computational Aspects

Computational Complexity of TSID

TSID solves LSP at each loop (embedded optimization, as MPC).

Computational Complexity of TSID

TSID solves LSP at each loop (embedded optimization, as MPC). →
Limited computation time (1-10 ms).

Computational Complexity of TSID

TSID solves LSP at each loop (embedded optimization, as MPC). → **Limited computation time** (1-10 ms).

For n_v DoFs, n_{va} motors, and n_f contact constraints:

- $n_v + n_{va} + n_f$ variables (≈ 70 for humanoid)
- $n_v + n_f$ equality constraints (≈ 40 for humanoid)
- $n_v + n_{va} + \frac{4}{3}n_f$ inequality constraints (*assuming friction cones approximated with 4-sided pyramids*)

Computational Complexity of TSID

TSID solves LSP at each loop (embedded optimization, as MPC). → **Limited computation time** (1-10 ms).

For n_v DoFs, n_{va} motors, and n_f contact constraints:

- $n_v + n_{va} + n_f$ variables (≈ 70 for humanoid)
- $n_v + n_f$ equality constraints (≈ 40 for humanoid)
- $n_v + n_{va} + \frac{4}{3}n_f$ inequality constraints (*assuming friction cones approximated with 4-sided pyramids*)

Computational cost dominated by Hessian (Cholesky) decomposition: $\mathcal{O}(n^3)$, with n = number of variables.

Computational Complexity of TSID

TSID solves LSP at each loop (embedded optimization, as MPC). → **Limited computation time** (1-10 ms).

For n_v DoFs, n_{va} motors, and n_f contact constraints:

- $n_v + n_{va} + n_f$ variables (≈ 70 for humanoid)
- $n_v + n_f$ equality constraints (≈ 40 for humanoid)
- $n_v + n_{va} + \frac{4}{3}n_f$ inequality constraints (*assuming friction cones approximated with 4-sided pyramids*)

Computational cost dominated by Hessian (Cholesky) decomposition: $\mathcal{O}(n^3)$, with n = number of variables.

QUESTIONS

- Can we solve it in 1 ms?
- Can we speed up computation?

Reformulating Optimization Problem

IDEA: Exploit problem structure to speed up computation.

Reformulating Optimization Problem

IDEA: Exploit problem structure to speed up computation.

Equality constraints have special structure:

$$\begin{bmatrix} J & 0 & 0 \\ M_u & -J_u^\top & -0 \\ M_a & -J_a^\top & -I \end{bmatrix} \begin{bmatrix} \dot{v} \\ f \\ \tau \end{bmatrix} = \begin{bmatrix} -j_v \\ -h_u \\ -h_a \end{bmatrix}$$

Reformulating Optimization Problem

IDEA: Exploit problem structure to speed up computation.

Equality constraints have special structure:

$$\begin{bmatrix} J & 0 & 0 \\ M_u & -J_u^\top & -0 \\ M_a & -J_a^\top & -I \end{bmatrix} \begin{bmatrix} \dot{v} \\ f \\ \tau \end{bmatrix} = \begin{bmatrix} -j_v \\ -h_u \\ -h_a \end{bmatrix}$$

Identity matrix is easy to invert $\rightarrow$ Easy to express τ as affine function of other variables.

$$\underbrace{\begin{bmatrix} \dot{v} \\ f \\ \tau \end{bmatrix}}_z = \underbrace{\begin{bmatrix} I & 0 \\ 0 & I \\ M_a & -J_a^\top \end{bmatrix}}_D \underbrace{\begin{bmatrix} \dot{v} \\ f \end{bmatrix}}_{\bar{z}} + \underbrace{\begin{bmatrix} 0 \\ 0 \\ h_a \end{bmatrix}}_d$$

Reformulating Optimization Problem

Original problem:

$$\underset{z}{\text{minimize}} \quad \|Az - a\|^2$$

$$\text{subject to} \quad Bz \leq b$$

$$\begin{bmatrix} J & 0 & 0 \\ M_u & -J_u^\top & -0 \\ M_a & -J_a^\top & -I \end{bmatrix} \begin{bmatrix} \dot{v} \\ f \\ \tau \end{bmatrix} = \begin{bmatrix} -j_v \\ -h_u \\ -h_a \end{bmatrix}$$

Reformulating Optimization Problem

Original problem:

$$\begin{aligned} & \underset{z}{\text{minimize}} \quad \|Az - a\|^2 \\ & \text{subject to} \quad Bz \leq b \end{aligned}$$
$$\begin{bmatrix} J & 0 & 0 \\ M_u & -J_u^\top & -0 \\ M_a & -J_a^\top & -I \end{bmatrix} \begin{bmatrix} \dot{v} \\ f \\ \tau \end{bmatrix} = \begin{bmatrix} -j_v \\ -h_u \\ -h_a \end{bmatrix}$$

Use $z = D\bar{z} + d$ to reformulate as [4]:

$$\begin{aligned} & \underset{\bar{z}}{\text{minimize}} \quad \|AD\bar{z} + Ad - a\|^2 \\ & \text{subject to} \quad BD\bar{z} \leq b - Bd \end{aligned}$$
$$\begin{bmatrix} J & 0 \\ M_u & -J_u^\top \end{bmatrix} \begin{bmatrix} \dot{v} \\ f \end{bmatrix} = \begin{bmatrix} -j_v \\ -h_u \end{bmatrix}$$

Reformulating Optimization Problem

Original problem:

$$\begin{aligned} & \underset{z}{\text{minimize}} \quad \|Az - a\|^2 \\ & \text{subject to} \quad Bz \leq b \end{aligned}$$
$$\begin{bmatrix} J & 0 & 0 \\ M_u & -J_u^\top & -0 \\ M_a & -J_a^\top & -I \end{bmatrix} \begin{bmatrix} \dot{v} \\ f \\ \tau \end{bmatrix} = \begin{bmatrix} -j_v \\ -h_u \\ -h_a \end{bmatrix}$$

Use $z = D\bar{z} + d$ to reformulate as [4]:

$$\begin{aligned} & \underset{\bar{z}}{\text{minimize}} \quad \|AD\bar{z} + Ad - a\|^2 \\ & \text{subject to} \quad BD\bar{z} \leq b - Bd \end{aligned}$$
$$\begin{bmatrix} J & 0 \\ M_u & -J_u^\top \end{bmatrix} \begin{bmatrix} \dot{v} \\ f \end{bmatrix} = \begin{bmatrix} -j_v \\ -h_u \end{bmatrix}$$

Removed n_{va} variables and n_{va} equality constraints!

So far $y(x, u) \in \mathbb{R}^m$.

From Euclidian Spaces to Lie Groups

So far $y(x, u) \in \mathbb{R}^m$.

What if $y(x, u) \in SE(3)$? (very common in practice)

From Euclidian Spaces to Lie Groups

So far $y(x, u) \in \mathbb{R}^m$.

What if $y(x, u) \in SE(3)$? (very common in practice)

SOLUTION Represent $SE(3)$ elements using homogeneous matrices $y \in \mathbb{R}^{4 \times 4}$ and redefine error function:

$$e(q, t) = \log(y^*(t)^{-1}y(q)),$$

where $\log \triangleq$ inverse operation of matrix exponential (i.e. exponential map): transforms displacement into twist.



W. Decré, R. Smits, H. Bruyninckx, and J. De Schutter.

Extending iTaSC to support inequality constraints and non-instantaneous task specification.

In *IEEE International Conference on Robotics and Automation (ICRA)*, 2009.



A. Del Prete.

Joint Position and Velocity Bounds in Discrete-Time Acceleration / Torque Control of Robot Manipulators.

IEEE Robotics and Automation Letters, 3(1), 2018.



A. Del Prete, N. Mansard, F. Nori, G. Metta, and L. Natale.

Partial Force Control of Constrained Floating-Base Robots.

In *Intelligent Robots and Systems (IROS 2014), IEEE International Conference on*, 2014.



A. Herzog, N. Rotella, S. Mason, F. Grimmering, S. Schaal, and L. Righetti.

Momentum control with hierarchical inverse dynamics on a torque-controlled humanoid.

Autonomous Robots, 40(3):473–491, 2016.



M. Mistry, J. Buchli, and S. Schaal.

Inverse dynamics control of floating base systems using orthogonal decomposition.

2010 IEEE International Conference on Robotics and Automation, (3):3406–3412, may 2010.



L. Righetti, J. Buchli, M. Mistry, and S. Schaal.

Inverse dynamics control of floating-base robots with external constraints: A unified view.

2011 IEEE International Conference on Robotics and Automation, pages 1085–1090, may 2011.



S. Rubrecht, V. Padois, P. Bidaud, M. Broissia, and M. Da Silva Simoes.

Motion safety and constraints compatibility for multibody robots.

Autonomous Robots, 32(3):333–349, 2012.